

Ottimizzazione

Nicolò Giuliani

Indice

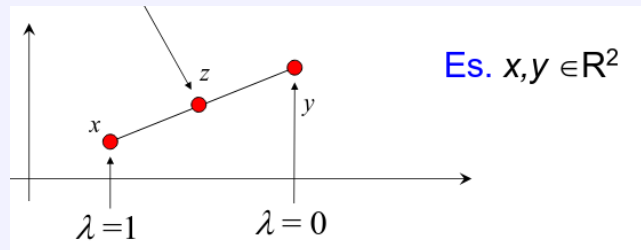
1	Modelli di Problemi di Ottimizzazione	2
1.1	Intro	2
1.1.1	Preliminari	2
1.1.2	Problemi di ottimizzazione	3
1.2	Modelli	7
1.2.1	Problema dello zaino	8
1.2.2	Problema MST	8
1.2.3	Problema TSP	9
1.2.4	Problema Min Cardinality Machine Scheduling	10
1.2.5	Selezione di sottoinsiemi	10
1.2.6	Set Covering Problem	10
1.2.7	Progetto di reti	12
1.2.8	Costo fisso di produzione	12
1.2.9	Ordinamento di lavori su macchine	13
1.2.10	Min Makespan Machine Scheduling	13
1.2.11	min Funzioni convesse	14
1.2.12	Modello PLI	15
1.2.13	Altri modelli PLI	15
1.2.14	Bin Packing Problem (BPP)	16
1.3	GNU	17
1.3.1	Caratteristiche avanzate	18
2	Problemi di ottimizzazione su reti	19
2.1	Problemi di Flusso su Rete	19
2.1.1	Generalita'	19
2.1.2	Definizione e Modelli Matematici	22
2.2	Il Problema del Flusso Massimo: Algoritmi	23
2.2.1	Algoritmo di Ford-Fulkerson	24
2.2.2	Algoritmo di Edmonds-Karp	25
2.2.3	Algoritmo di Golberg-Tarjan	25
2.3	Il Problema del Flusso di Costo Minimo: Algoritmi	27
2.4	Problemi di Accoppiamento	29
3	Algoritmi per PL e PLI	31
3.1	Geometria Della Programmazione Lineare	31
3.2	Dualità, Direzioni Ammissibili e di Crescita	35
3.3	L'algoritmo del Simplexso	36
3.4	La tecnica Branch-and-Bound	36

1 Modelli di Problemi di Ottimizzazione

1.1 Intro

1.1.1 Preliminari

Definizione: z è **combinazione convessa** di x, y se $\exists \lambda \in [0, 1]$ t.c. $z = \lambda x + (1 - \lambda)y$

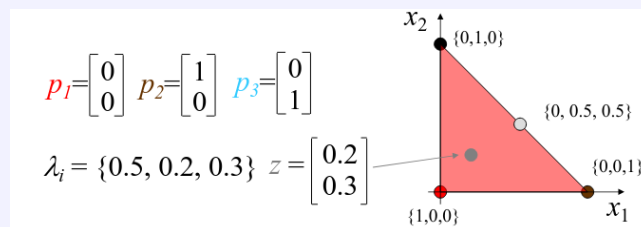


Definizione: z è **combinazione convessa** di K punti $p_1, p_2, \dots, p_K \in \mathbb{R}^n$

$$z = \sum_{i=1, K} \lambda_i p_i \quad \text{con } \lambda_i \geq 0$$

e

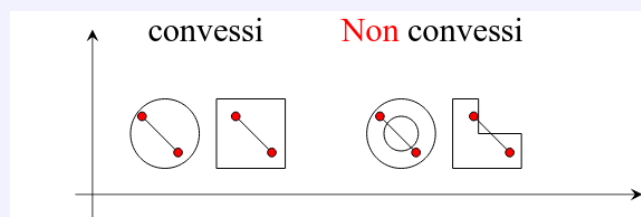
$$\sum_{i=1, K} \lambda_i = 1$$



Definizione $F \subseteq \mathbb{R}^n$ è un **insieme convesso** se

$$\forall x, y \in F \text{ e } \forall \lambda \in [0, 1]$$

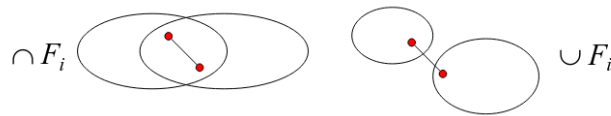
$$z = \lambda x + (1 - \lambda)y \in F$$



Proprietà degli insiemi convessi:

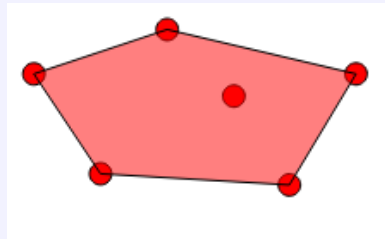
- $\mathbb{R}^n$ è convesso

- Se abbiamo F_i convessi allora $F = \bigcap F_i$ è convesso



Definizione: z è **vertice** di un insieme convesso F se e solo se non è combinazione convessa di altri punti di F

Definizione: dato un insieme di punti $P = \{p_1, \dots, p_K\} \subset \mathbb{R}^n$ si dice **chiusura convessa** di P ($\text{conv}(P)$) il più piccolo insieme che contiene P .



Definizione: Dato $F \subseteq \mathbb{R}^n$ convesso, allora

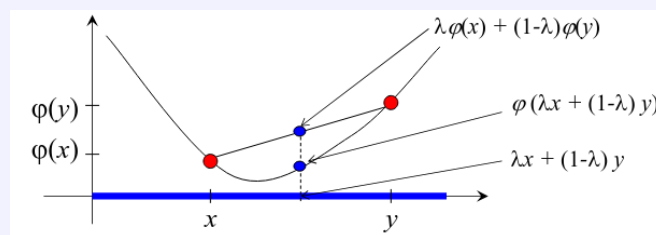
$$\phi : F \rightarrow \mathbb{R}$$

è convessa se:

$$\forall x, y \in F, \forall \lambda \in [0, 1]$$

si ha allora:

$$\phi(\lambda x + (1 - \lambda)y) \leq \lambda \phi(x) + (1 - \lambda)\phi(y)$$



1.1.2 Problemi di ottimizzazione

Un problema di ottimizzazione cerca la risposta che dipende da parametri e variabili. Un problema P viene descritto dai suoi parametri, variabili e i loro legami; e la descrizione delle soluzioni che devono avere.

Un'istanza del problema P si ottiene specificando valori concreti per tutti i parametri (NON VARIABILI!)

Problema	Istanza
$ax^2 + bx + c = 0$	$5x^2 - 6x + 1 = 0$
$Ax \leq 0$	$\begin{cases} x + y = 7 \\ x - y = 1 \end{cases}$

Possiamo dire che

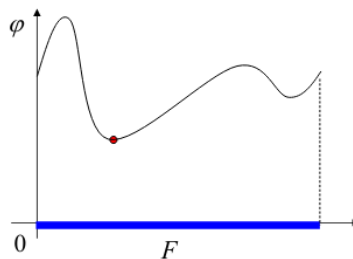
- $x = (x_1, \dots, x_n) \in F'$ vettore di **variabili decisionali**
- $F \subseteq F'$ insieme **soluzioni ammissibili**
- $c : F \rightarrow \mathbb{R}$ funzione **costo**

Per esempio vogliamo trovare il minimo con

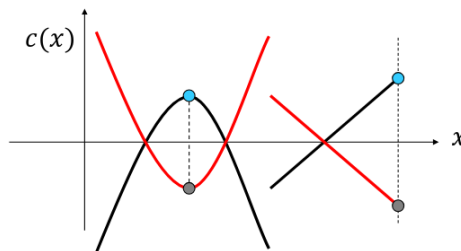
$$(P) \quad \min c(x) : x \in F$$

ovvero trovare $x^* \in F$ (ottimo globale) tale che

$$z(P) = c(x^*) \leq c(x) \forall x \in F$$

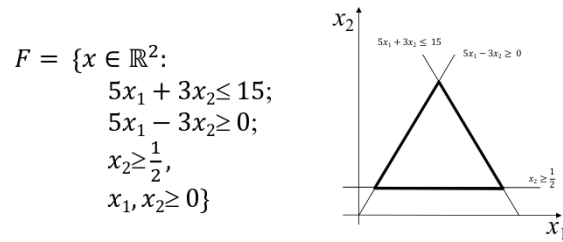


I problemi di ottimizzazione possono essere sia di minimizzazione che massimizzazione. Dato $(P) \quad \min c(x) : x \in F$, il problema $(P') \quad -\max -c(x) : x \in F$ è equivalente a (P) . Ossia $z(P) = z(P')$, i due problemi hanno lo stesso insieme di soluzioni ottime.



Regione ammissibile La regione ammissibile di F può essere definita come

- esplicitamente: specificando le proprietà di $x \in F$
- implicitamente: attraverso equazioni e disequazioni



Vedremo come sia più conveniente avere solo una tipologia di forma (solo disequazioni/equazioni, ecc). Dato un modello in forma generale possiamo costruire un nuovo modello con la forma desiderata. Per esempio

$$\min\{c(x) : x \in F\} = -\max\{-c(x) : x \in F\}$$

- Da disequazione a equazione:

$$\begin{aligned} -x_1 \leq d_1 &\rightarrow x_1 + x_s = d_1 \text{ con } x_s \text{ variabile } \mathbf{slack} \\ -x_1 \geq d_1 &\rightarrow x_1 - x_s = d_1 \text{ con } x_s \text{ variabile } \mathbf{surplus} \end{aligned}$$

- Da equazione a disequazione

$$a_j^T x = d_i \rightarrow \begin{cases} a_j^T x \geq d_j \\ -a_j^T x \geq -d_j \end{cases}$$

Le trasformazioni possono aumentare i vincoli o le variabili del modello rispetto a quello originale.

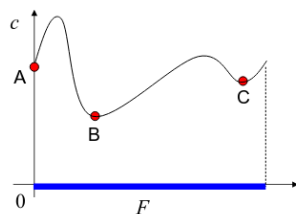
Segno delle variabili Generalmente le variabili sono vincolate ($x_j \geq 0$ o $x_j \leq 0$), nel caso non avessimo i vincoli ci si può sempre ricondurre ad un modello con variabili tutte ≥ 0

$$x_j \leq 0 \rightarrow y_i = -x_j \geq 0$$

Soluzioni Abbia quattro casi possibili:

- Problema impossibile: ($F = \emptyset$), allora si assume $z(P) = +\infty$
- Problema inferiormente (superiormente) illimitato: scelto un numero M abbiamo una soluzione $x \in F$ t.c. $c(x) \leq M$ (o $\geq M$)
- Il problema ha valore ottimo finito $-\infty < z(P) < \infty$ ma non ha soluzione ottima (non esiste $x \in F$ t.c. $c(x) = z(P)$)
- Il problema ha valore ottimo finito ed ammette soluzione ottima

Minimi Locali e Globali Non è detto che esista x^* ($F = \emptyset$) o che sia unica. Possono esistere ottimi (minimi) locali e globali



(P) richiede di trovare almeno un ottimo globale

La risoluzione di un problema di ottimizzazione richiede l'impiego di un metodo di calcolo (numerico e iterativo) codificato mediante un algoritmo.

- Algoritmo **esatto** per P : dato un'istanza di P fornisce in output la soluzione ottima x^* di P (se esiste)
- Algoritmo **euristico** per P : determina una qualsiasi soluzione $x \in F$ attraverso una approssimazione (minimo/massimo)
 - Dato ϵ -approssimazione allora x è approssimato se $c(x) \leq \epsilon$.

$$\text{Errore assoluto : } \epsilon_P = |c(x) - z(P)|$$

$$\text{Errore relativo : } R_P = \frac{\epsilon_P}{|z(P)|} = \frac{|c(x) - z(P)|}{|z(P)|}$$

Alcune volte anche calcolare l'errore diventa un problema se $z(P)$ non è noto. Allora risolviamo un nuovo problema approssimando quello originale.

Dato P definiamo **rilassamento** di P un problema del tipo

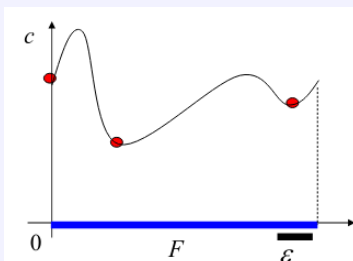
$$\bar{P} = \min\{\bar{c}(x) : x \in \bar{F}\}$$

t.c.

- $z(\bar{P}) \leq z(P)$
- $F \subseteq \bar{F}$ e $\forall x \in \bar{F}, \bar{c}(x) \leq c(x)$
- Se la soluzione ottima $\bar{x}^*$ di $\bar{P}$ soddisfa $\bar{x}^* \in F$ e $\bar{c}(\bar{x}^*) = c(\bar{x}^*)$ allora è ottima anche per P

Ovviamente gli algoritmi numerici non hanno una visione completa di c e F ma valuta $c(x)$ solo per un intervallo limitato di $x \in F$. Generalmente gli algoritmi di ottimizzazione sono iterativi, la soluzione converge verso una soluzione ottima (locale) con iterazioni molto alte o quando si raggiunge una approssimazione desiderata.

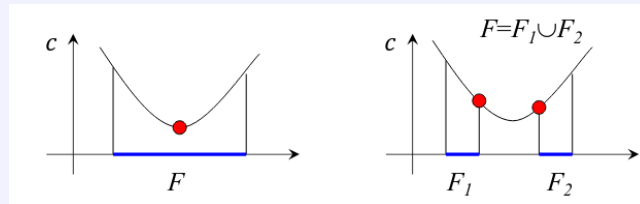
Definizione: $y \in F$ è **ottimo locale** se $\exists$ un interno $N \subseteq F$ tale che $c(y) \leq c(x) \forall x \in N$



N è **esatto** se un ottimo locale rispetto ad N è ottimo globale

(Es. $N \equiv F$)

Teorema: Dato $P(F, c)$ con $F \subseteq \mathbb{R}^n$ convesso e c convessa, l'intorno euclideo $N_\epsilon(y) = \{x \in F : \|y - x\| \leq \epsilon\}$ è **esatto** $\forall \epsilon > 0$.
 Ottimo locale $\equiv$ ottimo globale



Dimostrazione: OC2026-2-Intro_Ottimizzazione - Ver.2.0.pdf, pagina 40.

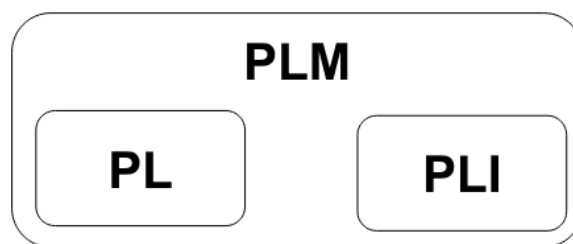
1.2 Modelli

Definito il problema dobbiamo classificarlo su un certo tipo. Attraverso un modello favoriamo l'astrazione e similitudini. Ci occuperemo di programmazione lineare:

Un problema di programmazione lineare (PL, LP) è un problema di ottimizzazione definito da

- numero finito di variabili decisionali $n \in \mathbb{N}$: $x = (x_1, \dots, x_n) \in \mathbb{R}^n$
- funzione obiettivo $f : \mathbb{R}^n \rightarrow \mathbb{R}$ nella forma $f(x) = c(x)$
- insieme m di vincoli lineari in una delle seguenti forme $ax = b$, $ax \leq b$, $ax \geq b$ con $a \in \mathbb{R}^n$, $b \in \mathbb{R}$

Alcune volte dobbiamo assumere $z \in \mathbb{Z}^n$ vettore delle soluzioni (di numeri interi). Allora siamo in caso di programmazione lineare intera (PLI, ILP) o programmazione lineare mista (PLM, MILP) se solo una parte delle variabili sono intere.



Possiamo definire un problema PL nella seguente forma

$$\min\{cx \mid Ax \leq b\}$$

con $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$.

Variabili booleane: condizioni binarie (0,1), spesso vengono usate come variabili ausiliarie per che stabiliscono il verificarsi o meno di condizioni specifiche.

1.2.1 Problema dello zaino

Dato uno zaino di capacità b e un insieme $E = \{1, 2, \dots, n\}$ di n oggetti caratterizzato da peso a_i e profitto c_i . Determinare il sottoinsieme $S \subseteq E$ t.c. profitto massimo e peso totale inferiore a b .

$$F = \{S \subseteq E : \sum_{i \in S} a_i \leq b\}$$

$$c(S) = \sum_{i \in S} c_i$$

Le soluzioni sono tutti i sottoinsiemi di E t.c. $2^{|E|} = 2^n$.

La soluzione ottimale è

$$S^* = \min\{c(S) : S \in F\}$$

Selezione:

$$\forall i = 1, \dots, n \quad x_i = \begin{cases} 1 & \text{se } i \text{ selezionato } (\in S^*) \\ 0 & \text{altrimenti} \end{cases}$$

$$\max c(x) = \sum_{i=1}^n c_i x_i$$

KP:

$$\sum_{i=1}^n a_i x_i \leq b \quad x_i \in 0, 1$$

Min-cost Spanning Tre

Dato un grafo non orientato $G = (N, A)$, con $|N| = n$, $|A| = m$ e $\forall a \in A \ c_a \geq 0$. Vogliamo cercare il grafo parziale $G' = (N, A')$ connesso e aciclico di costo minimo

$$F = \{T \subseteq A : (N, T) \text{ è uno } ST \text{ di } G\}$$

$$c(T) = \sum_{a \in A} c_a$$

La soluzione ottima è $T^* = \min\{c(T) : T \in F\}$.

Selezione:

$$\forall a \in A, \quad x_a = \begin{cases} 1 & \text{se } a \text{ selezionato } (\in T^*) \\ 0 & \text{altrimenti} \end{cases}$$

1.2.2 Problema MST

$$\forall a \in A, \quad x_a = \begin{cases} 1 & \text{se } a \text{ selezionato } (\in T^*) \\ 0 & \text{altrimenti} \end{cases}$$

$$\min c(x) = \sum_{a \in A} c_a x_a$$

MST:

$$\sum_{a \in A(N', N'')} x_a \geq 1 \quad \forall (N', N'') \quad x_a \in \{0, 1\} \quad a \in A$$

1.2.3 Problema TSP

Traveling salesman problem

$$F = \{P \subseteq A : (N, P) \text{ è ciclo Hamiltoniano di } G\}$$

$$c(P) = \sum_{(i,j) \in P} c_{i,j}$$

Dovendo visitare ogni nodo una sola volta P dovrà contenere esattamente due archi incidenti in ciascuna città : $\sum_{j:(i,j) \in A} = 2$ Soluzione ottima:

$$P^* = \min\{c(P) : P \in F\}$$

Selezione:

$$\forall (i, j) \in A, \quad x_{ij} = \begin{cases} 1 & \text{se } (i, j) \text{ selezionato } (\in P^*) \\ 0 & \text{altrimenti} \end{cases}$$
$$\min c(x) = \sum_{(i,j) \in A} c_{ij} x_{ij}$$

TSP:

$$\sum_{(i,j) \in A(N', N'')} x_{ij} \geq 1 \quad \forall (N', N'') \quad x_{ij} \in 0, 1 \quad (i, j) \in A$$

Notiamo come MST è un rilassamento di TSP!

Relazioni logiche tra variabili

Le relazioni logiche tra operatori hanno natura logica. Possiamo modificare le relazioni logiche attraverso vincoli lineari.

- **Negazione:** $y = \neg x \Rightarrow y = 1 - x$
- **Implicazione:** $x \rightarrow y \Rightarrow y \geq x$ (se $x = 0$ y è libera)
- **Intersezione (AND):** $z = (x \wedge y) \Rightarrow \begin{cases} z \leq x \\ z \leq y \\ z \geq x + y - 1 \end{cases}$
- **Unione (OR):** $z = (x \vee y) \Rightarrow \begin{cases} z \geq x \\ z \geq y \\ z \leq x + y \end{cases}$

Vincoli di assegnamento Spesso dobbiamo modellare associazioni tra entità del problema.

$$\forall (i, j) \quad x_{ij} = \begin{cases} 1 & \text{se oggetto } i \text{ assegnato a luogo } j \\ 0 & \text{altrimenti} \end{cases}$$

- **Semi-assegnamento:** ogni oggetto può essere assegnato ad 1 solo luogo

$$\sum_{j=1}^m x_{ij} = 1 \quad i = 1, \dots, n$$

- Assegnamento completo: ogni oggetto può essere assegnato ad 1 solo luogo, ad ogni luogo un solo oggetto

- caso $m = n$
 $\sum_{j=1}^m x_{ij} = 1 \quad i = 1, \dots, n \quad \sum_{i=1}^n x_{ij} = 1 \quad j = 1, \dots, m$
- caso $m > n$ (luoghi non utilizzati):
 $\sum_{j=1}^m x_{ij} = 1 \quad i = 1, \dots, n \quad \sum_{i=1}^n x_{ij} \leq 1 \quad j = 1, \dots, m$
- caso $m < n$ (lavori non assegnati)
 $\sum_{j=1}^m x_{ij} \leq 1 \quad i = 1, \dots, n \quad \sum_{i=1}^n x_{ij} = 1 \quad j = 1, \dots, m$

Ordinamento di lavori su macchine

Dati n lavori, il loro tempo di lavoro d_i e m macchine identiche. Dobbiamo assegnare i lavori alle macchine senza sovrapposizione e minimizzando il numero di macchine usate.

1.2.4 Problema Min Cardinality Machine Scheduling

Due lavori possono essere lavorati sulla stessa macchina se gli intervalli di lavorazione sono disgiunti. Se gli intervalli di lavorazione sono sovrapposti due lavori non possono essere eseguiti sulla stessa macchina.

$$\begin{aligned}
 \min c(y) &= \sum_{j=1}^m y_j \\
 \text{(MCMS)} \quad &\sum_{j=1}^m x_{ij} = 1 \quad i = 1, \dots, n \\
 &x_{ij} + x_{hj} \leq 1 \quad i = 1, \dots, n-1; \\
 &\quad \quad \quad h \in S(i); j = 1, \dots, m \\
 &x_{ij} \leq y_j \quad i = 1, \dots, n; j = 1, \dots, m \\
 &x_{ij} \in \{0,1\} \quad i = 1, \dots, n; j = 1, \dots, m \\
 &y_j \in \{0,1\} \quad j = 1, \dots, m
 \end{aligned}$$

1.2.5 Selezione di sottoinsiemi

N insieme di elementi, $F = F_1, \dots, F_n$ famiglia di suoi sottoinsiemi. Dobbiamo trovare un insieme di sottoinsiemi D di costo minimo che soddisfi i vincoli.

1.2.6 Set Covering Problem

D richiede di selezionare un sottoinsieme di F_j .

$$x_j = \begin{cases} 1 & \text{se } j \in D \\ 0 & \text{altrimenti} \end{cases} \quad j = 1, \dots, m$$

Funzione obbiettivo:

$$\min \sum_{j=1}^m c_j x_j$$

Set Covering Problem

ogni elemento $i \in N$ deve essere in almeno un sottoinsieme di D .

$$\sum_{j=1}^m a_{ij}x_j \geq 1 \quad i = 1, \dots, n$$

$$\min c(x) = \sum_{j=1}^m c_j x_j$$

SCP:

$$\sum_{j=1}^m a_{ij}x_j \geq 1 \quad i = 1, \dots, n$$

$$x_j \in \{0, 1\} \quad j = 1, \dots, m$$

Set Partitioning Problem

ogni elemento $i \in N$ deve essere esattamente in un sottoinsieme di D .

$$\sum_{j=1}^m a_{ij}x_j \geq 1 \quad i = 1, \dots, n$$

$$\min c(x) = \sum_{j=1}^m c_j x_j$$

SPP:

$$\sum_{j=1}^m a_{ij}x_j = 1 \quad i = 1, \dots, n$$

$$x_j \in \{0, 1\} \quad j = 1, \dots, m$$

Set Packing Problem

ogni elemento $i \in N$ deve essere al più in un sottoinsieme di D .

$$\sum_{j=1}^m a_{ij}x_j \leq 1 \quad i = 1, \dots, n$$

- problema di min: $D = \emptyset$
- problema di max:

$$\max c(x) = \sum_j 1^m c_j x_j$$

SPkP

$$\sum_{j=1}^m a_{ij}x_j \leq 1 \quad i = 1, \dots, n$$

$$x_j \in \{0, 1\} \quad j = 1, \dots, m$$

Variabili a valori discreti

Spesso le variabili sono vincolate ad assumere valori in un insieme che è $\{0, 1\}, N, [\dots]$. Per esempio x deve appartenere ad un insieme $\{v_1, \dots, v_k\}$ di valori reali.

Introduciamo k variabili binarie $y_i \in \{0, 1\}$ t.c.

$$\sum_{i=1}^k y_i = 1 \quad x = \sum_{i=1}^k v_i y_i$$

1.2.7 Progetto di reti

I collegamenti della rete sono descritti da un grafo orientato $G(V, A)$ con b_i come quantità massima per arco.

Per ogni nodo della rete il flusso portato dagli archi uscenti deve essere = al flusso entrante più la quantità prodotta nel nodo.

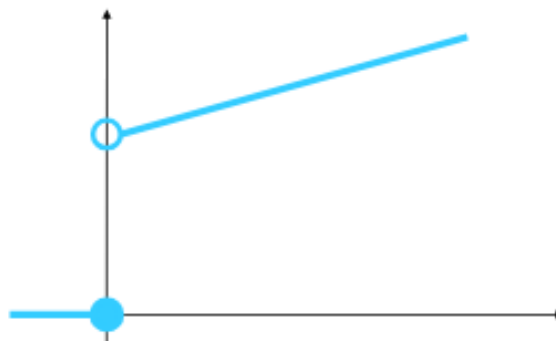
Minima quantità positiva

Quando x rappresenta un livello di produzione lo possiamo definire $\{0\} \cup [l, u]$. (0 rappresenta l'assenza di produzione). Introduciamo $y \in \{0, 1\}$ che ci indica la presenza di produzione

$$ly \leq x \quad x \leq uy$$

1.2.8 Costo fisso di produzione

Funzione con carico fisso, problema di mix di produzione. Se non produciamo prodotto ($x_j = 0$) allora non abbiamo costi, se produciamo ($x_j > 0$) allora abbiamo un costo fisso F_j più un costo variabile $p_j x_j$.



$$y_j = \begin{cases} 1 & \text{se } x_j > 0 \\ 0 & \text{se } x_j = 0 \end{cases}$$

$$\min \sum_{j=1}^n F_j y_j + p_j x_j$$

$$Ax \geq b$$

$$x \geq 0$$

$$y \in \{0, 1\}^n$$

Validita' del vincolo:

- se $x_j > 0$ allora $y_j = 1$ e il vincolo $x_j \leq +\infty$
- se $x_j = 0$ allora $y_j = 0$ o $y_j = 1$

oppure

- se $y_j = 0$ allora $x_j = 0$
- se $y_j = 1$ allora $x_j \geq 0$ ($\leq +\infty$)

Quindi cerchiamo

$$\begin{aligned} \min \sum_{j=1}^n F_j y_j + p_j x_j \\ Ax \geq b \\ x_j \leq M y_j \\ x_j \geq 0 \\ y_j \in \{0, 1\} \end{aligned}$$

1.2.9 Ordinamento di lavori su macchine

Variante di MCMS, dati n lavori, m macchine identiche. Assegnare i lavori alle macchine senza sovrapposizione e minimizzando il tempo massimo di completamento dei lavori sulle macchine.

$$\forall(i, j) \quad x_{ij} = \begin{cases} 1 & \text{se lavoro } i \text{ assegnato a macchina } j \\ 0 & \text{altrimenti} \end{cases} \quad i = 1, \dots, n \quad j = 1, \dots, m$$

Ogni lavoro deve essere assegnato ad una macchina

$$\sum_{j=1}^m x_{ij} = 1 \quad i = 1, \dots, n$$

Il tempo di completamento di una macchina j è la somma dei tempi di lavorazione dei lavori assegnati a essa:

$$t_j = \sum_{i=1}^n d_i x_{ij} \quad j = 1, \dots, m$$

Il tempo massimo di completamento è $t_{\max} = \max\{t_j \mid j = 1, \dots, m\}$, che può essere definito da m vincoli lineari:

$$t_j \geq \sum_{i=1}^n d_i x_{ij} \quad j = 1, \dots, m$$

1.2.10 Min Makespan Machine Scheduling

Anche noto come $P||C_{\max}$.

$$\min_t \sum_{j=1}^m x_{ij}$$

MMMS:

$$\begin{aligned} \sum_{j=1}^m x_{ij} &\leq t \\ x_{ij} &\in \{0, 1\} \quad i = 1, \dots, n \quad j = 1, \dots, m \end{aligned}$$

Valore Assoluto

Dobbiamo rappresentare $|g(x)|$.

Vincoli: $|g(x)| \leq b$ puo' essere espresso dalla coppia di vincoli $g(x) \leq b$ e $-g(x) \leq b$.

Funzione obbiettivo: $z = \min |g(x)|$ puo' essere calcolato risolvendo separatamente $z' = \max\{f(x) \mid x \in F\}$ e $z'' = \max\{f(x) \mid x \in F\}$, e definiamo $z = \max\{z', z''\}$.

Funzioni lineari a tratti

Come nel carico fisso si possono usare variabili binarie per rappresentare funzioni lineari a tratti.

$$y_1 = \begin{cases} 1 & \text{se } x \in [a_1, a_2] \\ 0 & \text{altrimenti} \end{cases} \quad y_2 = \begin{cases} 1 & \text{se } x \in (a_2, a_3] \\ 0 & \text{altrimenti} \end{cases}$$

Poiche' x deve appartenere ad un solo intervallo, allora $y_1 + y_2 = 1$.

Introduciamo anche 2 variabili ausiliare quantitative:

$$z_1 = \begin{cases} x - a_1 & \text{se } x \in [a_1, a_2] \\ 0 & \text{altrimenti} \end{cases} \quad z_2 = \begin{cases} x - a_2 & \text{se } x \in (a_2, a_3] \\ 0 & \text{altrimenti} \end{cases}$$

Le variabili ausiliare sono completamente definite dai vincoli

$$0 \leq z_1 \leq (a_2 - a_1)y_1$$

$$0 \leq z_2 \leq (a_3 - a_2)y_2$$

$$y_1 + y_2 = 1$$

$$y_1, y_2 \in \{0, 1\}$$

Funzioni convesse

Una funzione f lineare ad n tratti e' convessa se e' continua e la derivata non crescente.

1.2.11 min Funzioni convesse

$$\begin{aligned} \min & b_1 + c_1 a_1 + \sum_{i=1}^n z_i \\ & 0 \leq z_i \leq a_{i+1} - a_i \quad i = 1, \dots, n-1 \end{aligned}$$

Se il valore ottimo e' $\bar{x}$ si avra':

$$z_i = a_{i+1} - a_i \quad i = 1, \dots, h$$

$$z_h = \bar{x} - a_h \quad i = h$$

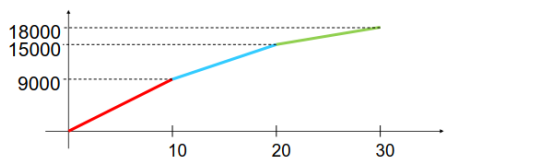
$$z_i = 0 \quad i = h+1, \dots, n$$

Analogamente vale per le max funzioni concave.

1.2.12 Modello PLI

Funzioni a tratti

Giornali		TV	
n. annunci	Nuovi utenti per annuncio	n. annunci	Nuovi utenti per annuncio
1-10	900	1-5	10000
11-20	600	6-10	5000
21-30	300	11-15	2000



Si possono utilizzare variabili binarie per rappresentare funzioni a tratti, per indicare se le variabili decisionali sono nella prima, seconda, terza fascia. Vincoli di tipo logico come per il costo fisso.

$$\begin{aligned}
 \max & 900G_1 + 600G_2 + 300G_3 + 10000T_1 + 5000T_2 + 2000T_3 \\
 & G_1 + G_2 + G_3 + 10T_1 + 10T_2 + 10T_3 \leq 150 \\
 & G_1, G_2, G_3 \leq 10 \\
 & T_1, T_2, T_3 \leq 5 \\
 & G_1, G_2, G_3, T_1, T_2, T_3 > 0 \text{ INTERE}
 \end{aligned}$$

1.2.13 Altri modelli PLI

Variante KP: KP-bounded

Il generico oggetto i e' disponibile in p_i esemplari. Puo' essere scelto p_i volte

$$\forall i = 1, \dots, n \quad x_i \geq 0 \text{ intera} = \text{n. di copie } i \text{ scelte}$$

$$\max c(x) = \sum_{i=1}^n c_i x_i$$

KP-b:

$$\begin{aligned}
 & \sum_{i=1}^n a_i x_i \leq b \\
 & 0 \leq x_i \leq p_i \text{ interi} \quad i = 1, \dots, n
 \end{aligned}$$

Variante KP: Subset-sum Problem

Il generico oggetto i ha profitto c_i e peso $a_i = c_i$. Dato un insieme di n numeri selezionare un sottoinsieme di numeri di somma massima e non eccedente a b . (Taglio di barre e minimizzazione

scarto)

$$\forall i = 1, \dots, n \quad x_i = \begin{cases} 1 & \text{se } i \text{ selezionato } (\in S^*) \\ 0 & \text{altrimenti} \end{cases}$$

$$\max c(x) = \sum_{i=1}^n c_i x_i$$

SSP:

$$\sum_{i=1}^n c_i x_i \leq b$$

$$x_i \in \{0, 1\} \quad i = 1, \dots, n$$

Problema KP multiplo (MKP)

Modello MKP

n oggetti, c_i profitto, a_{ij} peso su risorsa j . m contenitori di capacità b_j . Un oggetto può andare al più in un contenitore, ogni contenitore può contenere più oggetti.

Impaccare nei contenitori un sottoinsieme di oggetti avente massimo profitto in modo che la somma dei pesi degli oggetti inseriti in ogni contenitore non superi la corrispondente capacità b_i .

$$\forall i, j \quad x_{ij} = \begin{cases} 1 & \text{se } i \text{ inserito nel contenitore } j \\ 0 & \text{altrimenti} \end{cases}$$

$$\max c(x) = \sum_{i=1}^n \sum_{j=1}^m c_i x_{ij}$$

$$\sum_{j=1}^m x_{ij} \leq 1 \quad i = 1, \dots, n$$

MKP:

$$\sum_{i=1}^n a_{ij} x_{ij} \leq b_j \quad j = 1, \dots, m$$

$$x_{ij} \in \{0, 1\} \quad i = 1, \dots, n \quad j = 1, \dots, m$$

1.2.14 Bin Packing Problem (BPP)

n oggetti, a_i peso, ∞ contenitori di capacità b . Impaccare nei contenitori un sottoinsieme di oggetti avente massimo profitto in modo che la somma dei pesi degli oggetti inseriti in ogni contenitore non superi la corrispondente capacità b .

Modello PLI

$$\forall i, j = 1, \dots, n \quad x_{ij} = \begin{cases} 1 & \text{se } i \text{ inserito nel contenitore } j \\ 0 & \text{altrimenti} \end{cases}$$

$$\forall j = 1, \dots, n \quad y_j = \begin{cases} 1 & \text{se contenitore } j \text{ utilizzato} \\ 0 & \text{altrimenti} \end{cases}$$

Modello BPP

$$\begin{aligned} \min \sum_{j=1}^n y_j \\ \sum_{j=1}^m x_{ij} = 1 \quad i = 1, \dots, n \end{aligned}$$

BPP:

$$\begin{aligned} \sum_{i=1}^n a_i x_{ij} &\leq b y_j \quad j = 1, \dots, n \\ x_{ij} &\in \{0, 1\} \quad i = 1, \dots, n \quad j = 1, \dots, n \\ y_j &\in \{0, 1\} \quad j = 1, \dots, n \end{aligned}$$

Variante: Vector Packing (VPP)

Oltre al peso ogni oggetto i ha un volume v_i ed ogni contenitore ha anche una capacità massima di volume V .

Impaccare nei contenitori un sottoinsieme di oggetti avente massimo profitto in modo che la somma dei pesi e dei volumi degli oggetti inseriti in ogni contenitore non superi le corrispondenti capacità b e V .

$$\begin{aligned} \sum_{i=1}^n a_i x_{ij} &\leq b y_j \quad j = 1, \dots, n \\ \sum_{i=1}^n v_i x_{ij} &\leq V y_j \quad j = 1, \dots, n \end{aligned}$$

Variante: Bin con capacità variabile

Il generico contenitore j ha una capacità b_j e un costo c_j .

Impaccare nei contenitori un sottoinsieme di oggetti avente massimo profitto in modo che la somma dei pesi degli oggetti inseriti in ogni contenitore non superi le corrispondenti capacità b_j e minimizzando il costo totale dei contenitori utilizzati.

$$\begin{aligned} \min \sum_{j=1}^n c_j y_j \\ \sum_{i=1}^n a_i x_{ij} \leq b_j y_j \quad j = 1, \dots, n \end{aligned}$$

1.3 GNU

GNU MathProg è un linguaggio in cui è possibile scrivere dei modelli di programmazione lineare, per poi venire risolti con GLPK (*GNU Linear Programming Kit*).

Prendiamo in considerazione il problema

$$\begin{aligned} \max_{x_1, x_2} \quad & 80x_1 + 100x_2 \\ \text{s.t.} \quad & 2x_1 + 4x_2 \leq 80 \\ & 80x_1 + 60x_2 \leq 2400 \end{aligned}$$

Possiamo risolvere il problema con GNU:

```
# decision variables
var x1>=0; # product A
var x2>=0; # product B

maximize pounds: 80*x1 + 100*x2; # objective

# constraints
s.t. hours:      2*x1 + 4*x2 <= 80;
s.t. material: 80*x1 + 60*x2 <= 2400;
solve;

# Output code
printf "The optimal production per day is:\n";
printf " %.1f pallets of product A,\n",x1;
printf " %.1f pallets of product B \n",x2;
printf "This will return a profit of %.2f pounds\n",pounds;
end;
```

Che ci dara' come output:

```
The optimal production per day is:
 24.0 pallets of product A,
  8.0 pallets of product B
This will return a profit of 2720.00 pounds
```

1.3.1 Caratteristiche avanzate

Prima abbiamo utilizzato l'istruzione **printf** per presentare i risultato del processo di ottimizzazione. Pero' possiamo scrivere direttamente il valore con **display**:

```
display x1, x2;
display pounds;
```

che ci restituirà:

```
Display statement at line 16
x1.val = 24
x2.val = 8
Display statement at line 17
pounds.val = 2720
```

Tramite la keyword **param** è possibile definire dei parametri numerici, in modo da evitare di inserire il dato numerico direttamente nel modello.

```
param m, integer, > 0; # number of agents
param n, integer, > 0; # number of tasks
```

Si possono usare le keyword `set`, assieme alle istruzioni **sum** e **for**, in modo da parametrizzare ulteriormente la definizione dei modelli.

```
set I := 1..m; # set of agents
set J := 1..n; # set of tasks

param c{i in I, j in J}, >= 0; /* cost of allocating task j to agent
    i */

var x{i in I, j in J}, >= 0; /* x[i,j] = 1 means task j is assigned
    to agent */
```

Possiamo anche combinarlo con *printf* per stampare tabelle

```
printf "Agent Task Cost\n";
printf{i in I} "%5d %5d %10g\n", i, sum{j in J} j * x[i,j], sum{j in
    J} c[i,j] * x[i,j];
printf "-----\n";
```

Per avere in output

Agent	Task	Cost
1	1	13
2	8	8
3	7	13
4	5	12
5	2	6
6	6	16
7	4	3
8	3	5

2 Problemi di ottimizzazione su reti

2.1 Problemi di Flusso su Rete

2.1.1 Generalita'

Molti problemi pratici sono modellati naturalmente da grafi.

Definizione: Una **Rete** e' rappresentata da un grafo, generalmente orientato (diretto) $G = (N, A)$ in cui

- i vertici (nodi) $i \in N$, con $|N| = n$ sono punti di **ingresso**, **uscita**, **trasferimento**
- gli archi $(i, j) \in A$ con $|A| = m$ sono i **collegamenti** tra i nodi

Una **soluzione** e' rappresentata da un flusso di oggetti tra i nodi attraverso gli archi della rete e puo' essere

- discreto (camion o pacchetti)
- continuo (fluido o di corrente)

Definizione: Ad ogni nodo $i \in N$ e' associata un valore reale b detto **sbilanciamento** che puo' essere

- Positivo:
 - Il nodo i e' un nodo di uscita dalla rete e viene detto **destinazione, pozzo, nodo di output**
 - b_i e' detto domanda di i
 - $D \subset N = \{i \in N : b_i > 0\}$
- Negativo:
 - Il nodo i e' un nodo di ingresso della rete e viene detto **origine, sorgente, nodo di input**
 - b_i e' detto offerta di input
 - $O \subset N = \{i \in N : b_i < 0\}$
- Nullo:
 - Il nodo i e' detto di **trasferimento**
 - $T \subset N = \{i \in N : b_i = 0\}$

$$N = O \cup D \cup T$$

Definizione: Ad ogni arco $(i, j) \in A$ sono associati:

- Un **costo** c_{ij} che indica quanto costa per unita' di flusso utilizzare o attraversare il collegamento
- Una **capacita inferiore** l_{ij} che indica un limite inferiore alla quantita' di flusso che puo' influire attraverso il collegamento
- Una **capacita' superiore** u_{ij} che indica un limite superiore alla quantita' di flusso che puo' influire attraverso il collegamento

Nei problemi di flusso, una **soluzione** non è altro che un flusso, ossia un assegnamento di valori reali agli archi di una rete $G = (N, A)$.

Il costo di un flusso non è nient'altro che il costo complessivo di tutti i flussi presenti nella rete:

$$\sum_{(i,j) \in A} c_{ij} x_{ij}$$

Come vincoli abbiamo:

- **Domanda ed offerta globale si equivalgono:**

$$\sum_{i \in D} b_i = - \sum_{i \in O} b_i \iff \sum_{i \in N} b_i = 0$$

- **Il flusso deve essere ammissibile:**

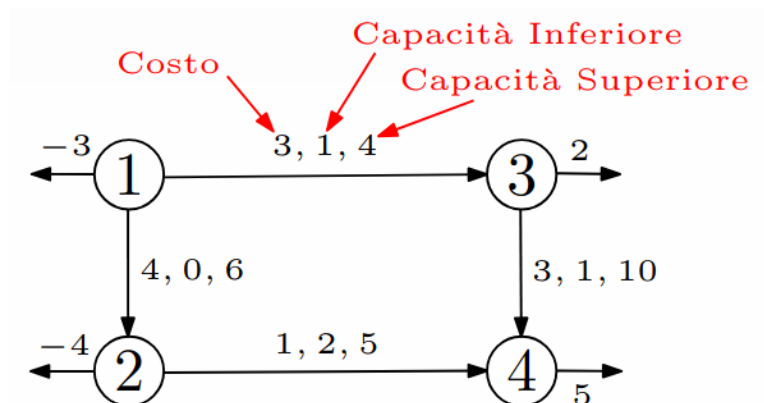
$$l_{ij} \leq x_{ij} \leq u_{ij} \quad \forall i, j \in A$$

- Il flusso si conserva:

$$\sum_{(i,j) \in BS(i)} x_{ij} - \sum_{(i,j) \in FS(j)} x_{ij} = b_i \quad \forall i \in N$$

dove

- $BS(i) = \{(k,i) | (k,i) \in A\}$: detto *backward star* di i
- $FS(i) = \{(i,k) | (i,k) \in A\}$: detto *forward star* di i



Ipotesi semplificative: capacità inferiore nulla Per semplificare problema e notazione supponiamo che $l_{ij} = 0 \quad \forall (i,j) \in A$, e' sempre possibile farlo:

- Data una rete G esiste sempre una rete H equivalente ad G che abbia capacita' inferiori nulle.
 1. si sottrae la quantita' l_{ij} a b_{ij} e a u_{ij}
 2. si aggiunge la quantia' l_{ij} a b_i
 3. alla funziona obbiettivo bisognera' aggiungere

$$\sum_{(i,j) \in A} c_{ij} x_{ij}$$

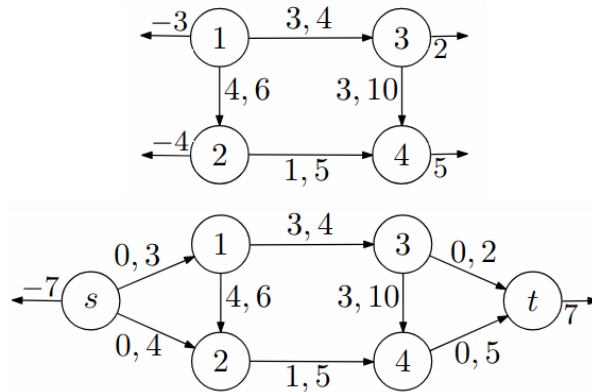
Ovvero al flusso x_{ij} in H corrisponde al flusso $x_{ij} + l_{ij}$ in G .

Ipotesi semplificativa: una sola sorgente e pozzo Spesso, nel progetto di algoritmi per problemi di flusso, conviene assumere che vi sia una sola sorgente e un solo pozzo.

- Si può costruire una rete equivalente:
 - Aggiungendo due nodi fittizi, uno corrispondente all'unica sorgente, e l'altro all'unico pozzo.
 - Aggiungendo archi fittizi dalla sorgente a ciascuna delle sorgenti della rete di partenza. Tali archi avranno costo nullo, e capacità superiore pari all'inverso dello sbilanciamento della sorgente.
 - Aggiungendo archi fittizi da ciascuno dei pozzi della rete di partenza al pozzo. Tali archi avranno anch'essi costo nullo, ma capacità superiore pari allo sbilanciamento del pozzo

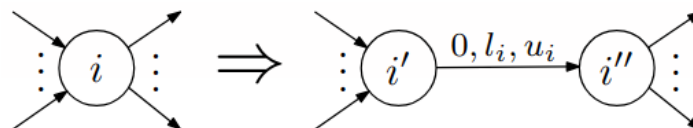
- Lo sbilanciamento dell'unica sorgente sarà uguale alla somma degli sbilanciamenti delle sorgenti. Similmente per il pozzo.

Ipotesi semplificativa: una sola sorgente e pozzo



Capacità sui nodi Alcune volte è utile imporre che anche i nodi (e non solo gli archi) abbiano delle capacità

- si sdoppia ogni nodo i con capacità in due nodi i', i'' in modo che:
 - tutti gli archi entrati in i entrano in i'
 - tutti gli archi uscenti da i partano da i''
 - Si aggiunge un arco fittizio (i', i'') con costo nullo capacità inferiore l_i e capacità superiore u_i



2.1.2 Definizione e Modelli Matematici

Problema del Flusso di Costo Minimo (MCF)

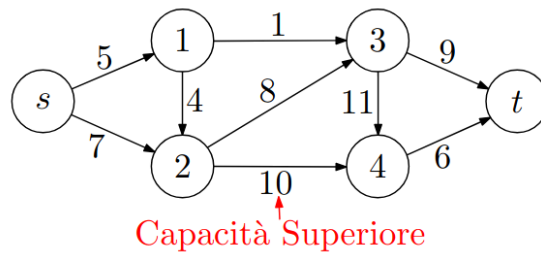
$$\begin{aligned} \min & cx \\ & 0 \leq x \leq u \quad Ex = b \end{aligned}$$

con c vettore costi, u vettore capacità superiori, E matrice di incidenza sui nodi e archi, b vettore degli sbilanciamenti.

Il **problema di flusso massimo** (o maximum flow, MF) è un problema di ottimizzazione su reti che può essere visto come una restrizione di MCF.

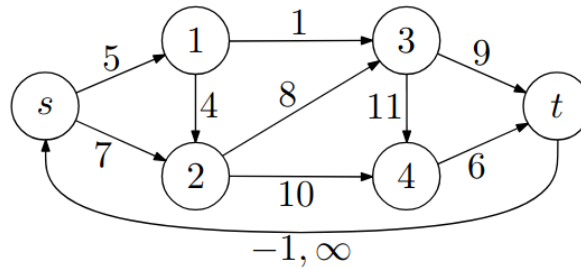
Ciò che cambia è la funzione obiettivo: non vogliamo minimizzare i costi, ma piuttosto **massimizzare** i flussi.

$$\begin{aligned}
& \max v \\
& \sum_{(j,s) \in BS(s)} x_{js} + v = \sum_{(s,j) \in FS(s)} x_{sj} \\
& \sum_{(j,s) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} = 0, \quad i \in N - \{s, t\} \\
& \sum_{(j,s) \in BS(t)} x_{ji} = \sum_{(t,j) \in FS(t)} x_{tj} + v \\
& 0 \leq x_{ij} \leq u_{ij}, \quad (i, j) \in A
\end{aligned}$$



Il problema MF può essere visto come un caso particolare di MCF:

- costi nulli
- sbilanciamenti nulli
- Si aggiunge però un arco fittizio da t a s con costo -1 e capacità infinita



2.2 Il Problema del Flusso Massimo: Algoritmi

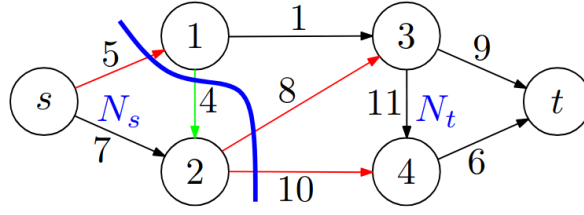
Definizione: Un taglio in una rete $G = (N, A)$ è dato da una coppia (N', N'') di sottoinsiemi di N t.c. $N' \cap N'' = \emptyset$ e $N' \cup N'' = N$.

Un (s, t) -taglio in una rete G è un taglio (N_s, N_t) dove $s \in N_s$ e $t \in N_t$.

- Dato un (s, t) -taglio in $G = (N, A)$ indichiamo con $A^+(N_s, N_t)$ e $A^-(N_s, N_t)$ i seguenti sottoinsiemi di A :

$$A^+(N_s, N_t) = \{(i, j) \in A \mid i \in N_s \wedge j \in N_t\}$$

$$A^-(N_s, N_t) = \{(i, j) \in A \mid i \in N_t \wedge j \in N_s\}$$



Lemma: Per ogni (s, t) -taglio (N_s, N_t) e ogni flusso ammissibile x con valore v :

1. **Flusso del taglio:** $v = \sum_{(i,j) \in A^+(N_s, N_t)} x_{ij} - \sum_{(i,j) \in A^-(N_s, N_t)} x_{ij}$; si indica con $x(N_s, N_t)$
2. **Capacità del taglio:** $v \leq \sum_{(i,j) \in A^+(N_s, N_t)} u_{ij}$; si indica con $u(N_s, N_t)$

$$v = x(N_s, N_t) \leq u(N_s, N_t)$$

Cammini Aumentanti Un cammino aumentante P in una rete G rispetto a x non è nient'altro che un cammino semplice e orientato da s a t in G_x .

Dato un cammino aumentante P rispetto a x , definiamo la capacità di P rispetto a x come:

$$\theta(P, x) = \min\{\min\{u_{ij} - x_{ij} | (i, j) \in P^+\}, \min\{x_{ij} | (j, i) \in P^-\}\}$$

Dato un flusso x , un cammino P in G_x e un reale θ , definiamo $x(P, \theta)$ il flusso definito come segue:

$$(x(P, \theta)) = \begin{cases} x_{ij} + \theta & \text{se } (i, j) \in P^+ \\ x_{ij} - \theta & \text{se } (j, i) \in P^- \\ x_{ij} & \text{altrimenti} \end{cases}$$

2.2.1 Algoritmo di Ford-Fulkerson

1. $x \leftarrow 0$
2. Costruisci G_x e determina se esiste un cammino aumentante P .
Se tale P non esiste, termina e restituisci x .
3. $x \leftarrow x(P, \theta(P, x))$
4. Ritorna al punto 2.

Lemma: Se x è ammissibile allora anche $x(P, \theta(P, x))$ è ammissibile.

Lemma: Se x è un flusso ammissibile massimo, allora G_x non ha cammini aumentanti.

Lemma: Se G_x non ha cammini aumentanti, allora esiste un taglio di capacità pari a v .

Teorema (Correttezza): Se l'algoritmo di Ford-Fulkerson termina, allora il flusso x è un flusso massimo.

Teorema: Il valore del massimo flusso è uguale alla minima capacità dei tagli.

Teorema: Se le capacità di G sono numeri interi, allora esiste almeno un flusso intero massimo.

2.2.2 Algoritmo di Edmonds-Karp

L'Algoritmo di Edmonds-Karp non è nient'altro che l'algoritmo di Ford-Fulkerson dove, però, la ricerca del cammino aumentante viene eseguita visitando in ampiezza (BFS) il grafo residuo G_x .

Notiamo che in questo modo i cammini aumentanti saranno sempre cammini di **lunghezza minima**.

Lemma: Se, durante l'esecuzione di EK, il flusso y è ottenuto da x tramite un'operazione di aumento del flusso in un cammino aumentante, allora per ogni nodo $i \in N$, vale che $\delta_x(s, i) \leq \delta_y(s, i)$.

Teorema: Il numero di iterazioni di EK è $O(NA)$, quindi la sua complessità è $O(NA^2)$.

$$\begin{aligned}\delta_y(s, i) &= \delta_y(s, j) + 1 \geq \delta_x(s, j) + 1 \\ &= \delta_x(s, i) + 2\end{aligned}$$

Da un momento in cui (i, j) diventa critico al successivo, la sua distanza da s aumenta di almeno 2. Siccome tale distanza non può essere superiore a $|N|$, il numero di volte in cui (i, j) può diventare critico è lineare in $|N|$.

2.2.3 Algoritmo di Goldberg-Tarjan

Una delle strade che ci permettono di scendere sotto la soglia di una complessità $O(NA^2)$ è quella che Goldberg e Tarjan intrapresero nel 1986 e che consiste nel rendere la costruzione del flusso massimo più **locale**. (questo in opposizione con FF ed EK, in cui ad ogni iterazione occorre procedere con un'analisi **globale**)

C'è un prezzo da pagare, ossia il fatto che durante l'esecuzione dell'algoritmo, non si lavora solo con flussi ammissibili, ma anche con **preflussi**.

Un **preflusso** è un vettore x t.c.

$$\begin{aligned}\sum_{(j,i) \in BS(i)} x_{ij} - \sum_{(i,j) \in FS(i)} x_{ij} &> 0, \quad i \in N - \{s, t\} \\ 0 \leq x_{ij} &\leq u_{ij}, \quad (i, j) \in A\end{aligned}$$

Un nodo si dice **attivo** se il suo eccesso è positivo, altrimenti si dice **bilanciato**.

$$e_i = \sum_{(j,i) \in BS(i)} x_{ij} - \sum_{(i,j) \in FS(j)} x_{ij}$$

L'idea centrale dell'algoritmo è quella di eliminare gli sbilanciamenti presenti nel preflusso corrente in modo *iterativo* e *locale*.

Lo sbilanciamento presente in un nodo i viene eliminato spostando parte del flusso in eccesso:

- In avanti (**push forward**), ossia attraverso un arco (i, j) , ogniqualvolta $x_{ij} < u_{ij}$
- Indietro (**push backward**), ossia attraverso un arco (j, i) , ogniqualvolta $x_{ij} > 0$

Non è sempre possibile effettuare *push*, altrimenti sarebbe molto difficile tenere la complessità dell'algoritmo sotto controllo. Si usa invece un sistema basato su **etichettature**.

Teorema: L'algoritmo di Goldberg e Tarjan è corretto, e la sua complessità in tempo è $O(N^2 A)$.

Algorithm 1 GOLDBERG-TARJAN(G, s, t)

```
1:  $x \leftarrow 0$ 
2: for all  $(s, j) \in FS(s)$  do
3:    $x_{sj} \leftarrow u_{sj}$ 
4: end for
5:  $d \leftarrow \text{ETICHETTATURAVALIDA}(G)$ 
6:  $d_s \leftarrow n$ 
7: if tutti i nodi (diversi da  $s$  e  $t$ ) sono bilanciati then
8:   return  $x$ 
9: end if
10: while esiste un nodo sbilanciato  $v$  do
11:   if esiste  $(v, j)$  ammissibile per  $v$  then
12:     PUSHFORWARD( $v, j$ )
13:   else if esiste  $(i, v)$  ammissibile per  $v$  then
14:     PUSHBACKWARD( $i, v$ )
15:   else
16:     RELABEL( $v$ )
17:   end if
18: end while
19: return  $x$ 
```

2.3 Il Problema del Flusso di Costo Minimo: Algoritmi

Uno **pseudoflusso** è un vettore x che soddisfa i vincoli di capacità, ossia tale che

$$0 \leq x_{ij} \leq u_{ij}, \quad (i, j) \in A$$

Se x è uno pseudoflusso, definiamo sbilanciamento di un nodo i rispetto a x la quantità

$$e_x(i) = \sum_{(j,i) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} - b_i$$

Possiamo anche vedere e_x come un vettore, detto vettore degli **sbilanciamenti**.

Dato uno pseudoflusso x , i nodi sbilanciati rispetto a x fanno parte di uno dei seguenti due insiemi:

$$O_x = \{i \in N | e_x(i) > 0\}$$

$$D_x = \{i \in N | e_x(i) < 0\}$$

I nodi in O_x sono detti **nodì con eccesso di flusso**, mentre quelli in D_x sono detti **nodì con difetto di flusso**.

- Se $O_x = D_x = \emptyset$ allora x è un flusso.

Lo **sbilanciamento complessivo** di x è definito come:

$$g(x) = \sum_{i \in O_x} e_x(i) = - \sum_{j \in D_x} e_x(j)$$

Un cammino P tra i e j in Gx verrà quindi detto cammino aumentante tra i e j

- I suoi archi possono essere partizionati in P^+ e P^-
- La sua capacità $\theta(P, x)$ è definita come al solito
- Un cammino aumentante tra i e i viene anche detto **ciclo aumentante**

Il **costo** di un cammino aumentante P è definito come:

$$c(P) = \sum_{(i,j) \in P^+} c_{ij} - \sum_{(i,j) \in P^-} c_{ij}$$

Teorema: Siano x e y due pseudoflussi qualunque. Allora esistono $k \leq n + m$ cammini aumentanti $P_1, \dots, P_k$, tutti per x , di cui al più m sono cicli, t.c.

$$z_1 = x$$

$$z_{i+1} = z_i \oplus \theta_i P_i \quad 1 \leq i \leq k$$

$$z_{k+1} = y$$

$$0 \leq \theta_i \leq \theta(P_i, z_i)$$

Inoltre, tutti i P_i hanno come estremi dei nodi in cui lo sbilanciamento di x è diverso da quello di y .

Lemma: Uno pseudoflusso (rispettivamente, un flusso ammissibile) è minimale (rispettivamente, ottimo) sse non esistono cicli aumentanti di costo negativo.

Teorema: Sia x uno pseudoflusso minimale e sia P un cammino aumentante rispetto ad x avente costo minimo tra tutti i cammini che uniscono un nodo di O_x ad un nodo di D_x . Allora, qualunque sia $\theta \leq \theta(x, P)$, abbiamo che $x(\theta, P) = x \oplus \theta P$ è ancora pseudoflusso minimale.

È abbastanza facile costruire uno pseudoflusso minimale x , se non si bada agli sbilanciamenti. Ad esempio, il flusso x definito come

$$x_{ij} = \begin{cases} 0 & \text{se } x_{ij} \geq 0 \\ u_{ij} & \text{altrimenti} \end{cases}$$

In questo modo i costi degli archi in G_x sono tutti non-negativi e quindi G_x non avrà cicli negativi (e in ultima analisi x sarà minimale).

Il costo computazionale di **ogni iterazione** è dominato dalla ricerca di un cammino minimo,

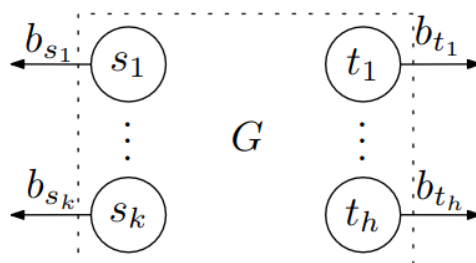
Algorithm 2 CAMMINI MINIMI SUCCESSIVI(G)

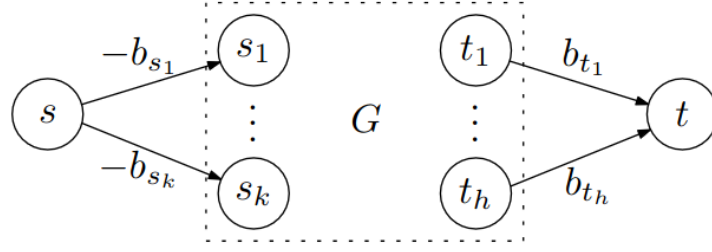
```

1:  $x \leftarrow \text{PSEUDOFLUSSOMINIMALE}(G)$ 
2: if  $g(x) = 0$  then
3:   return  $x$ 
4: end if
5: while true do
6:   Cerca un cammino di costo minimo  $P$  tra un nodo  $i \in O_x$  e  $j \in D_x$ 
7:   if  $P$  non esiste then
8:     return “il problema è vuoto”
9:   end if
10:   $x \leftarrow x(P, \min\{\theta(P, x), ex(i), -ex(j)\})$ 
11: end while
```

che possiamo eseguire in tempo $O(NA)$. La complessità sarà quindi nel caso peggiore $O(\bar{g}NA)$, **pseudopolinomiale** nella dimensione del grafo.

Costruire un Flusso Ammissibile Data una rete G , è possibile determinare se esiste un **flusso** ammissibile (ma non necessariamente ottimo) per essa? Basta risolvere il problema MF sulla rete ottenuta nel modo seguente:





Algorithm 3 CANCELLAZIONE CICLI(G)

```

1:  $x \leftarrow \text{FLUSSOAMMISSIBILE}(G)$ 
2: if  $x$  non esiste then
3:   return “il problema è vuoto”
4: end if
5: while true do
6:   Cerca un ciclo di costo negativo in  $G_x$ 
7:   if nessun ciclo trovato then
8:     return  $x$ 
9:   end if
10:  Sia  $C$  il ciclo trovato
11:   $x \leftarrow x(C, \theta(C, x))$ 
12: end while

```

Il costo di qualunque flusso ammissibile è compreso tra $-A\bar{u}\bar{c}$ e $A\bar{u}\bar{c}$, dove

$$\bar{u} = \max\{u_{ij} | (i, j) \in N\}$$

$$\bar{c} = \max\{c_{ij} | (i, j) \in N\}$$

La complessità dell'algoritmo sarà quindi **pseudopolinomiale**, ossia

$$O(NA) \cdot O(A\bar{u}\bar{c}) = O(NA^2\bar{u}\bar{c})$$

2.4 Problemi di Accoppiamento

Nei problemi di accoppiamento, si lavora con grafi bipartiti non orientati ossia con grafi nella forma $G = (O \cup D, A)$, dove:

- $O = \{1, \dots, n\}$ e' l'insieme dei **nodì di origine**
- $D = \{n+1, \dots, n+d\}$ e' l'insieme dei **nodì di destinazione**
- $A \subseteq O \times D$ e' l'insieme degli **archi** a ciascun dei quali e' associato un **costo**

Un **accoppiamento** per un grafo bipartito $G = (O \cup D, A)$ è un sottoinsieme M di A i cui archi non abbiano nodi in comune.

- Gli archi in M si dicono **interni**, mentre quelli in $A - M$ sono detti **esterni**.
- I nodi che compaiono in qualche arco di M si dicono **accoppiati**, gli altri nodi si dicono invece **esposti**.

M è detto **accoppiamento perfetto** sse non vi sono nodi esposti.

Il *costo* di un accoppiamento M è nient'altro che

$$c(M) = \sum_{(i,j) \in M} c_{ij}$$

Accoppiamento di Massima Cardinalità Si vuole determinare, semplicemente, l'accoppiamento di massima cardinalità.

Il problema può essere visto come un problema di flusso massimo con più sorgenti (i nodi in O) e più pozzi (i nodi in D).

- Le **capacità** saranno tutte pari a 1
- Ci interessano sono **flussi interi**
- Come sappiamo, una rete con molte sorgenti e molte destinazioni può essere poi tradotta in una rete con una sorgente e una destinazione.

Ogni cammino aumentante in G_M deve:

- Essere alternante, ossia consistere di archi interni, seguiti da archi esterni, seguiti da archi interni, e così via.
- Partire da un'origine esposta e arrivare ad una destinazione esposta.
- In altre parole, se $P_E = P - M$ e $P_I = M \cap P$ sono rispettivamente gli archi esterni e interni di un cammino aumentante P , allora $|P_E| - |P_I| = 1$.

La capacità $\theta(M, P)$ di un cammino aumentante P sarà sempre 1, e di conseguenza:

$$M \oplus (\theta(M, P))P = (M - P_I) \cup P_E$$

Otteniamo in questo modo un algoritmo del tutto simile a FF, ma in cui la ricerca del cammino aumentante può essere eseguita tramite una semplice procedura di visita.

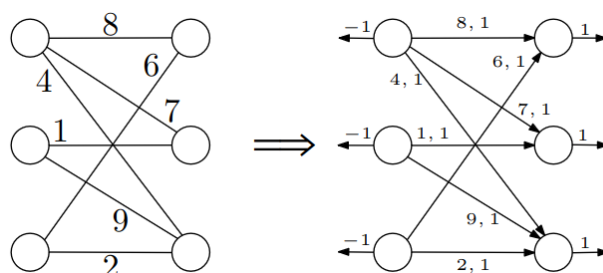
La complessità di quest'algoritmo, a differenza di quella di FF, sarà $O(mn)$, perché

$$U = \max\{c_{ij} | (i, j) \in A\} = 1$$

Accoppiamento di Costo Minimo Si vuole determinare l'accoppiamento di costo minimo tra tutti gli accoppiamenti perfetti.

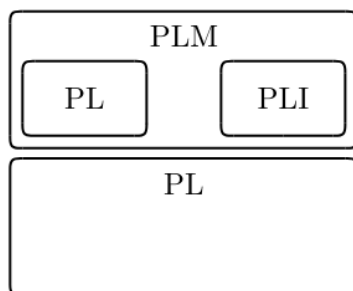
Seguendo lo stesso identico tipo di ragionamento, è possibile vedere una qualunque istanza del problema di accoppiamento di costo minimo come un'istanza di MCF.

In particolare, i cammini minimi aumentanti potrebbero essere visti come cammini esposti tra due vertici esposti, rispettivamente in O e in D .



3 Algoritmi per PL e PLI

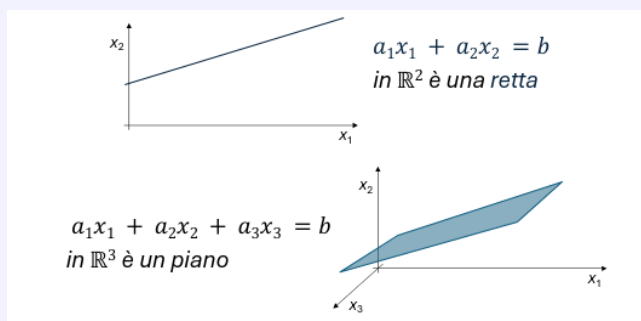
3.1 Geometria Della Programmazione Lineare



Lo spazio di ricerca, nei problemi PL, è in linea di principio infinito, però lo spazio di ricerca può essere in qualche caso ridotto ad un insieme finito, ossia l'insieme dei vertici del poliedro che definisce la regione ammissibile.

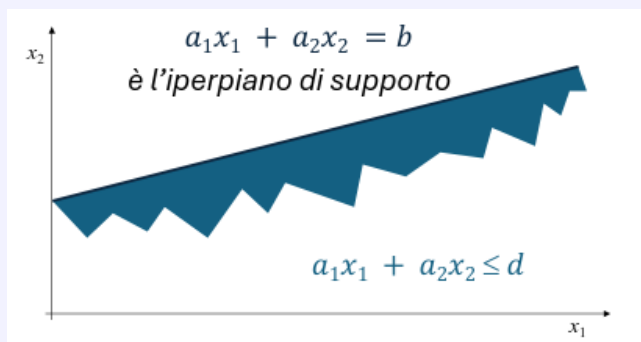
La domanda cui cercheremo di dare una risposta in questa prima sezione è la seguente: è possibile generalizzare quest'argomento al caso di problemi in $n > 2$ variabili?

Definizione Iperpiano: Insieme $\{x \in \mathbb{R}^n | ax = b\}$ delle soluzioni dell'equazione lineare $ax = b$ dove $a \in \mathbb{R}^n$ e $b \in \mathbb{R}$

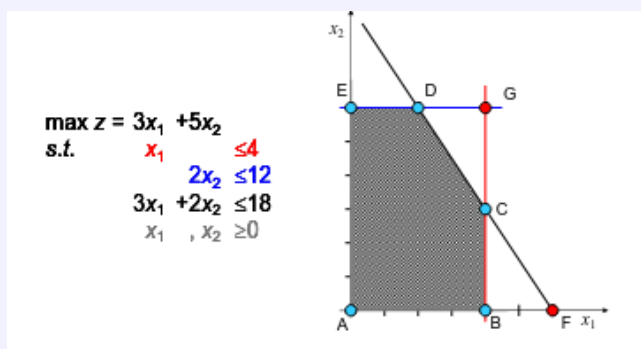


Definizione Semispazio: Insieme $\{x \in \mathbb{R}^n | ax \leq b\}$ delle soluzioni dell'equazione lineare $ax = b$ dove $a \in \mathbb{R}^n$ e $b \in \mathbb{R}$.

Un *iperpiano* è il confine del corrispondente semispazio.

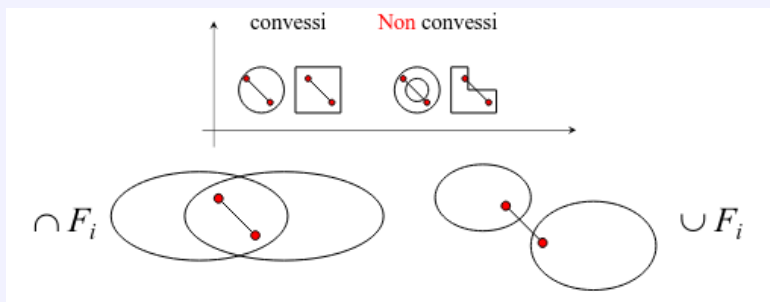


Definizione Poliedro: Intersezione P di un numero finito m di semispazi. Devono esistere una matrice $A \in \mathbb{R}^{m \times n}$ e un vettore $b \in \mathbb{R}$ tali che $P = \{x | Ax \leq b\}$.

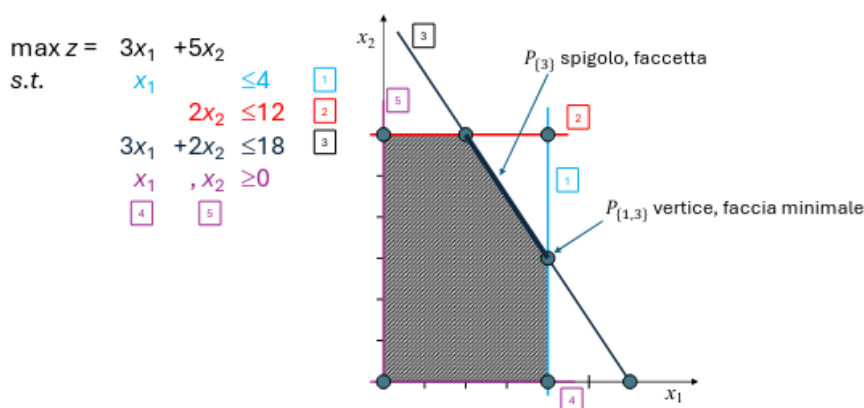


Definizione Insieme convesso: Un insieme $C \subseteq \mathbb{R}^n$ tale che tutti i punti che connettono $x, y \in C$ sono anche essi in C ossia

$$\forall x, y \in C \quad \forall a \in [0, 1] \quad ax + (1 - a)y \in C$$



Definizione Faccia: Se I è tale che P_I non è vuoto, chiamiamo il poliedro P_I faccia di P .



Soluzioni di Base Dato un poliedro $P = \{x | Ax \leq b\}$, supponiamo che $I = B$ sia t.c. A_B matrice quadrata e invertibile. Allora:

- B e' detta base
- A_B e' detta matrice di base

- $x_B = A_B^{-1}b_B$ e' detta soluzione di base

Una soluzione di base x_B t.c. $x_B \in P$ e' detta **ammissibile**, altrimenti **non ammissibile**.

E' facile rendersi conto che i vertici di P sono tutte e sole le sue soluzioni di base ammissibili.

Vincoli attivi Se $x \in P$ allora i vincoli che vengono soddisfatti come uguaglianze sono detti **attivi** in x . Indichiamo con $I(x)$ l'insieme degli indici dei vincoli attivi in x :

$$I(x) = \{i | A_i x = b_i\}$$

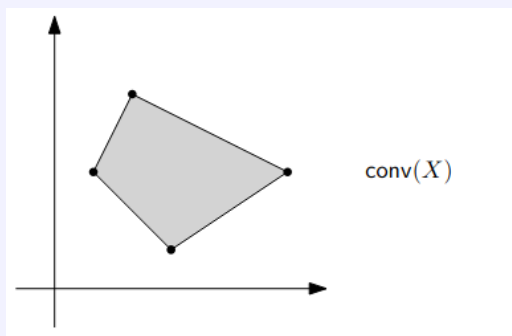
Per ogni $J \subseteq I(x)$ l'insieme P_J e' una faccia di P , $P_{I(x)}$ e' la faccia minimale tra esse.

Definizione Involuppi Convessi: I poliedri fin'ora li abbiamo rappresentati per facce, ma possono essere rappresentati anche per punti.

Dato un insieme di punti $X = \{x_1, \dots, x_S\} \subseteq \mathbb{R}^n$ l'**inviluppo convesso** di X e' definito come l'insieme

$$\text{conv}(X) = \{x = \sum_{i=1}^S \lambda_i x_i \mid \sum_{i=1}^S \lambda_i = 1 \wedge \lambda_i \geq 0\}$$

Per esempio per $X = \{(1, 3), (3, 1), (2, 5), (6, 3)\}$



Definizione Coni convessi: Un insieme $C \subseteq \mathbb{R}^n$ e' detto **cono** sse per ogni $x \in C$ e per ogni $a \in \mathbb{R}^+$ vale che $ax \in C$.

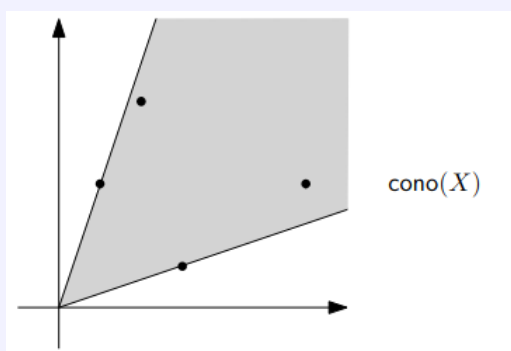
I coni che siano anche insiemi convessi (detti coni convessi) sono caratterizzabili equivalentemente come gli insiemi C tali che

$$x, y \in C \wedge \lambda, \mu \in \mathbb{R}^+ \Rightarrow \lambda x + \mu y \in C$$

Anche per i coni convessi esiste una rappresentazione basata sulle direzioni: dato un insieme $V = \{v_1, \dots, v_t\} \subset \mathbb{R}^n$, il **cono finitamente generato** da V è:

$$\text{cono}(V) = \left\{ v = \sum_{i=1}^t \mu_i v_i \mid \mu_i \in \mathbb{R}^+ \right\}$$

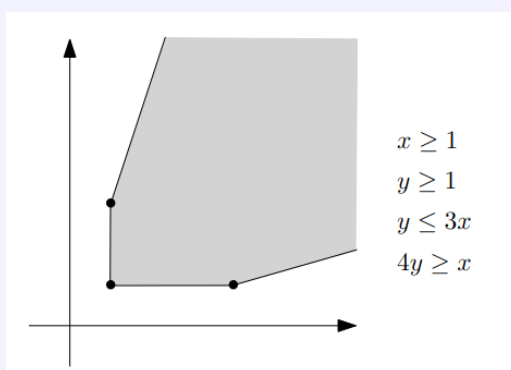
Per esempio per $X = \{(1, 3), (3, 1), (2, 5), (6, 3)\}$



Il Teorema di Motzkin (Decomposizione) Dati $X, Y \subseteq \mathbb{R}^n$, indichiamo con $X + Y$ il sottoinsieme di $\mathbb{R}^n$ definito ponendo

$$X + Y = \{x + y \mid x \in X \wedge y \in Y\}$$

Teorema (Motzkin): $P \subseteq \mathbb{R}^n$ è un poliedro sse esistono X, V finiti tali che $P = \text{conv}(X) + \text{cono}(V)$



Due Rappresentazioni

- Poliedri come intersezioni di semispazi
- Poliedri come somma di un politopo e di un cono

Le due rappresentazioni sono equivalenti (grazie al teorema di Motzkin) ma non hanno la stessa dimensione.

Teorema: Sia $P = \{x | Ax \leq b\}$ e siano $x_1, \dots, x_s, v_1, \dots, v_t \in \mathbb{R}^n$ tali che

$$P = \text{conv}(\{x_1, \dots, x_s\}) + \text{cono}(\{v_1, \dots, v_t\})$$

Allora il problema $\max\{cx | Ax \leq b\}$ ha ottimo finito sse $cv_j \leq 0$ per ogni $j \in \{1, \dots, t\}$. In tal caso esiste inoltre un $k \in \{1, \dots, s\}$ tale che x_k è una soluzione ottima.

3.2 Dualità, Direzioni Ammissibili e di Crescita

La teoria della dualità è una branca dell'algebra lineare che risulta estremamente utile nella costruzione degli algoritmi per PL.

La teoria della dualità si basa sulla definizione di un'involuzione (ossia di una funzione inversa di sé stessa) che mappa ogni problema PL nel suo duale.

- Primale: $\max\{cx | Ax \leq b\}$
- Duale: $\min\{yb | (yA = c) \wedge (y \geq 0)\}$

Proprietà Primale e Duale È abbastanza facile dimostrare che il duale del duale è il primale.

Teorema Debole di Dualità: Se $\bar{x}$ e $\bar{y}$ sono soluzioni ammissibili per il primale e il duale, rispettivamente, allora $c\bar{x} \leq \bar{y}b$.

Corollario:

- Se il primale è illimitato, allora il duale è impossibile (vuoto)
- Se $\bar{x}$ e $\bar{y}$ sono soluzioni ammissibili per il primale e il duale, rispettivamente, e $c\bar{x} = \bar{y}b$, allora $\bar{x}$ e $\bar{y}$ sono soluzioni ottime

Direzioni Ammissibili — I Data una coppia asimmetrica, consideriamo una soluzione ammissibile $\bar{x}$ per il primale, e chiediamoci se spostandoci lungo una direzione dell'iperspazio a partire da $\bar{x}$, si resta o meno nella regione ammissibile.

Un vettore $\xi \in \mathbb{R}^n$ è detto **Direzione Ammissibile** se esiste $\bar{\lambda} > 0$ t.c. $x(\lambda) = \bar{x} + \lambda\xi$ è ammissibile nel primale per ogni $\lambda \in [0, \bar{\lambda}]$.

Direzioni Ammissibili — II Il vettore ξ è direzione ammissibile per $\bar{x}$ sse $A_{I(\bar{x})}\xi \leq 0$.

Direzioni di Crescita Una direzione $\xi \in \mathbb{R}^n$ è una **direzione di crescita** per $\bar{x}$ spostamento λ lungo ξ fa crescere il valore della funzione obiettivo ossia se:

$$cs(\lambda) = c\bar{x} + \lambda c\xi > c\bar{x} \iff c\xi > 0$$

La nozione di direzione di crescita, dunque, non dipende dal punto $\bar{x}$!

3.3 L'algoritmo del Simplexso

Algorithm 4 Simplex Primal

```

1:  $N \leftarrow \{1, \dots, m\} \setminus B$ 
2:  $\bar{x} \leftarrow A_B^{-1}b_B$ 
3:  $\bar{y}_B \leftarrow c_B A_B^{-1}$ 
4:  $\bar{y}_N \leftarrow 0$ 
5: if  $\bar{y}_B \geq 0$  then
6:   return  $\bar{x}, \bar{y}$  ▷ soluzione ottima
7: end if
8:  $h \leftarrow \min\{i \in B \mid \bar{y}_i < 0\}$ 
9: Sia  $\xi$  la colonna di indice  $h$  in  $-(A_B^{-1})$ 
10: if  $A_i \xi \leq 0$  per ogni  $i$  then
11:   return  $\xi$  ▷ problema illimitato
12: end if
13:  $k \leftarrow \arg \min \left\{ \frac{b_i - A_i \bar{x}}{A_i \xi} \mid A_i \xi > 0, i \in N \right\}$ 
14:  $B \leftarrow (B \cup \{k\}) \setminus \{h\}$ 
15: goto 1

```

L'algoritmo lavora mantenendo i seguenti tre invarianti:

- B e' una **base ammissibile**
- $\bar{x}$ e' **soluzione ammissibile** per il problema primale

$$\max\{cx \mid Ax \leq b\}$$

mentre $\bar{y}A = c$.

- Di conseguenza, $\bar{y}$ è soluzione per il duale sse $\bar{y}_B > 0$

$$\max\{yb \mid yA = c, y \geq 0\}$$

Se, quindi $\bar{y}_B \geq 0$, allora l'algoritmo correttamente **termina**, restituendo $\bar{x}$ e $\bar{y}$ che sono soluzioni ottime per il primale e per il duale (riga 5.)

Se, invece, c'è un elemento di $\bar{y}$ strettamente negativo, allora non vale l'ottimalità. Cerchiamo quindi una direzione ammissibile e di crescita per $\bar{x}$.

3.4 La tecnica Branch-and-Bound

Finora ci siamo occupati della ricerca dell'ottimo in programmi lineari in cui tutte le variabili siano reali. Alla PLI si applica però una tecnica, detta branch-and-bound che, anche se esponenziale in tempo nel caso peggiore, permette in molti casi di evitare l'enumerazione esaustiva. Perche' la tecnica del branch-and-bound sia applicabile ad una certa classe di problemi di ottimizzazione (che assumiamo di minimo), devono valere due requisiti:

- **Rilassamento:** Deve essere possibile, in ogni momento, passare da un problema $\mathbb{P}$ ad un suo rilassamento $\mathbb{T} = \text{RELAX}(\mathbb{P})$, tipicamente più semplice da risolvere da un punto di vista computazionale. (In questo modo si possono facilmente calcolare limitazioni *inferiori* al valore ottimo di $\mathbb{P}$.)

$$\text{RELAX}(\min\{cx \mid Ax \leq b \wedge x \in \mathbb{Z}^n\}) = \min\{cx \mid Ax \leq b\}$$

- **Branching:** Deve esistere un modo per *partizionare* l'insieme delle soluzioni ammissibili di $\mathbb{P}$ ottenendo due sottoproblemi $\text{PARTITION}(\mathbb{P}) = (\mathbb{T}, \mathbb{Q})$.

$$\text{PARTITION}(\min\{cx \mid Ax \leq b \wedge x \in \mathbb{Z}^n\}) = \min\{cx \mid Ax \leq b \wedge x_i \geq n+1 \wedge x \in \mathbb{Z}^n\}$$

Algorithm 5 Branch and Bound per $\mathcal{P}$

```

1:  $S \leftarrow \{\mathcal{P}\}; \quad v^* \leftarrow \infty$ 
2: if  $S = \emptyset$  then
3:   return soluzione ottima  $x^*$  se definita
4: end if
5: Scegli un problema  $T \in S; \quad S \leftarrow S \setminus \{T\}$ 
6: if  $\text{RELAX}(T)$  è vuoto then
7:   goto 2
8: end if
9: if  $\text{RELAX}(T)$  è illimitato then
10:   $(Q, S') \leftarrow \text{PARTITION}(T)$ 
11:   $S \leftarrow S \cup \{Q, S'\}$ 
12:  goto 2
13: end if
14: Siano  $x$  e  $v$  la soluzione e il valore ottimo di  $\text{RELAX}(T)$ 
15: if  $v \geq v^*$  then
16:   goto 2
17: end if
18: if  $x$  è ammissibile per  $T$  e  $v < v^*$  then
19:   $v^* \leftarrow v; \quad x^* \leftarrow x$ 
20:  goto 2
21: end if
22: if  $x$  non è ammissibile per  $T$  e  $v < v^*$  then
23:   $(Q, S') \leftarrow \text{PARTITION}(T)$ 
24:   $S \leftarrow S \cup \{Q, S'\}$ 
25:  goto 2
26: end if

```
